

Q56) Difference between AVL Trees and Red-Black Trees

PAGE NO. : _____

DATE : ____/____/____

Introduction :

AVL trees and Red-Black trees are both self-balancing binary search trees used to maintain balanced height for efficient operations like searching, insertion, and deletion.

Definition (Short) :

- **AVL Tree :** A self-balancing BST where the height difference (balance factor) between left and right subtree is at most 1.
- **Red-Black Tree :** A self-balancing BST where each node is colored red or black and follows specific rules to maintain balance.

Difference :

Basis	AVL Tree	Red-Black Tree
Balancing Method	Strict balancing using height difference	Balancing using color properties (red/black)
Balance Condition	Balance factor = -1, 0, +1	Follows color rules (no two red nodes together, etc.)
Height	More strictly balanced (shorter height)	Less strictly balanced (slightly taller)
Rotations	More rotations required	Fewer rotations required
Search Operation	Faster searching due to better balance	Slightly slower than AVL
Insertion/Deletion	More complex and time-consuming	Simpler and faster
Usage	Used when searching is frequent	Used in libraries and systems (like maps, sets)

Explanation (Extra Lines) :

- AVL trees maintain strict balance, so they provide faster search operations.
- Red-Black trees allow a little imbalance but reduce the number of rotations, making insertion and deletion faster.

Conclusion :

Thus, AVL trees are more balanced and better for searching, while Red-Black trees are better for faster insertion and deletion with fewer rotations.

1) What is Data Structure?

Definition:

A Data Structure is a way of organizing and storing data in a computer's main memory so that it can be accessed and modified efficiently.

Explanation:

Data structures help in managing large amounts of data in an efficient manner. They make operations like searching, insertion, and deletion faster and easier. Choosing the right data structure improves the performance of a program.

Types of Data Structure:

→ Linear Data Structure – Elements are arranged in a sequential manner.
Examples: Array, Linked List, Stack, Queue

→ Non-Linear Data Structure – Elements are not arranged sequentially.

Examples: Tree, Graph

2) Types of Data Structures

Introduction :

Data structures are classified based on how data elements are arranged and connected with each other. The two main types are linear and non-linear data structures.

1. Linear Data Structure

In a linear data structure, elements are arranged in a sequential order. Each element is connected to its previous and next element.

Examples: Array, Linked List, Stack, Queue

2. Non-Linear Data Structure

In a non-linear data structure, elements are not arranged sequentially. Data is organized in a hierarchical or network form.

Examples: Tree, Graph

3) Operations on Data Structure

Introduction :

Operations on data structures are the basic actions performed on data to process and manage it efficiently.

Basic Operations :

1. Traversing

It means visiting each element of the data structure one by one in a systematic manner.

2. Searching

It is used to find a specific element in the data structure.

3. Insertion

It means adding a new element at a specific position in the data structure.

4. Deletion

It means removing an element from the data structure.

5. Sorting

It is the process of arranging data in a particular order (ascending or descending).

6. Merging

It means combining two data structures into one.

4) What is Algorithm?

Introduction:

An algorithm is used to solve a problem step by step in a logical manner.

Definition:

An Algorithm is a finite set of step-by-step instructions used to solve a problem or perform a specific task.

Characteristics of Algorithm:

- Finite – It must have a limited number of steps.
- Definite – Each step should be clear and unambiguous.
- Input – It can take zero or more inputs.
- Output – It should produce at least one output.
- Effective – Steps should be simple and executable.

Example:

Algorithm to add two numbers:

- ① Start
- ② Input two numbers
- ③ Add the numbers
- ④ Display the result
- ⑤ Stop

5) Difference between Time Complexity and Space Complexity

Introduction :

Time complexity and space complexity are used to measure the efficiency of an algorithm. They help us understand how well an algorithm performs in terms of time and memory.

Difference :

Basis	Time Complexity	Space Complexity
Definition	It is the amount of time taken by an algorithm to complete execution	It is the amount of memory used by an algorithm during execution
Focus	Focuses on how fast the algorithm runs	Focuses on how much memory is required
Measurement	Measured in terms of number of operations (time)	Measured in terms of memory space (bytes)
Goal	To reduce execution time	To reduce memory usage
Example	$O(n)$, $O(\log n)$, $O(n^2)$	$O(n)$, $O(1)$

Explanation (Extra lines for marks) :

- If an algorithm takes less time to run, it has better time complexity.
- If an algorithm uses less memory, it has better space complexity.
- Both are important for writing efficient programs.

6) Time-Space Tradeoff

Introduction :

In computer science, time and space are two important factors used to measure the efficiency of an algorithm. Sometimes, improving one factor may worsen the other.

Definition :

Time-Space Tradeoff means a situation where we can reduce the execution time of an algorithm by using more memory, or reduce memory usage by increasing execution time.

Explanation :

- If we want a program to run faster, we may use extra memory (space).
- If we want to save memory, the program may take more time to execute.
- So, there is always a balance between time and space.

Example :

- Using an extra array can make searching faster (less time, more space)
- Without extra memory, the same task may take more time

7) Define Array with example

Definition:

An Array is a collection of elements of the same data type stored in contiguous memory locations, where each element is accessed using an index.

Explanation:

- All elements in an array are of the same type (like all integers).
- Elements are stored in continuous memory.
- Each element has an index (position), starting from 0.

Example:

If we store marks of 5 students:

marks = [50, 60, 70, 80, 90]

Index	0	1	2	3	4
marks	50	60	70	80	90

marks[0] = 50

marks[1] = 60

marks[2] = 70

marks[3] = 80

marks[4] = 90

Conclusion:

Thus, an array is used to store multiple values of the same type in an organized and efficient way.

UNIT 2: Arrays

PAGE NO. : _____

DATE : ____/____/____

Q8) Operations on Array (Traversal, Insertion, Deletion)

Introduction:

Operations on an array are the basic actions performed to access and modify the elements of the array.

1. Traversal

Traversal means visiting each element of the array one by one.

Explanation:

- It is used to display or process all elements
- Elements are accessed using index (0, 1, 2, ...)

Example:

Array: [10, 20, 30]

Traversal $\rightarrow 10 \rightarrow 20 \rightarrow 30$

2. Insertion

Insertion means adding a new element at a specific position in the array.

Explanation:

- Elements are shifted to make space
- Can be inserted at beginning, middle, or end

Example:

Array: [10, 20, 30]

Insert 15 at position 1 $\rightarrow$ [10, 15, 20, 30]

3. Deletion

Deletion means removing an element from the array.

Explanation:

- Elements are shifted after deletion to fill the gap

Example:

Array: [10, 20, 30]

Delete 20 $\rightarrow$ [10, 30]

UNIT 2: Arrays

PAGE NO. : _____

DATE : ____/____/____

Q9) Linear Search

Introduction:

Searching is an important operation used to find a specific element in a data structure. Linear search is the simplest searching technique.

Definition:

Linear Search is a method of searching in which each element of the array is checked one by one until the desired element is found or the list ends.

Explanation:

- ① Start from the first element
- ② Compare each element with the target value
- ③ If match is found $\rightarrow$ element is found
- ④ If not $\rightarrow$ move to next element
- ⑤ Continue until element is found or array ends

Example:

Array: [10, 20, 30, 40]

Search = 30

Index	0	1	2	3
Array	10	20	30	40

Check $\rightarrow$ 10 $\times$ $\rightarrow$ 20 $\times$ $\rightarrow$ 30 ☒

Element found at position 2

UNIT 2: Arrays

PAGE NO. : _____

DATE : ____/____/____

Q10) Binary Search

Introduction:

Searching is an important operation in data structures.

Binary search is a fast searching technique used for sorted data.

Definition:

Binary Search is a searching method in which the array is divided into two halves repeatedly to find the desired element. It works only on sorted arrays.

Explanation:

- ① Start with the middle element of the array
- ② If the middle element is equal to the target $\rightarrow$ found
- ③ If the target is smaller $\rightarrow$ search in left half
- ④ If the target is greater $\rightarrow$ search in right half
- ⑤ Repeat until element is found or search ends

Example:

Array : [10, 20, 30, 40, 50]

Search = 30



Index	0	1	2	3	4
Array	10	20	30	40	50

$$\text{Middle index} = \frac{(0 + 4)}{2} = 2$$

Middle element = 30

Middle = 30 ☒

Element found at position 2

UNIT 2: Arrays

PAGE NO. : _____

DATE : ____/____/____

Q11) Difference: Linear Search vs Binary Search ★

Introduction:

Linear search and binary search are two methods used to find an element in an array. They differ in working method, speed, and requirements.

Basis	Linear Search	Binary Search
Definition	It is a searching technique in which each element is checked one by one until the desired element is found.	It is a searching technique in which the array is repeatedly divided into two halves to find the element.
Working Method	Starts from the first element and checks every element sequentially.	Starts from the middle element and reduces search area step by step.
Data Requirement	Works on both sorted and unsorted data.	Works only on sorted data.
Speed	Slower, especially for large data.	Faster and more efficient.
Time Complexity	$O(n)$ (linear time)	$O(\log n)$ (logarithmic time)
Comparisons	May require checking all elements in worst case.	Requires fewer comparisons due to division.
Implementation	Simple and easy to implement.	Slightly more complex to implement.
Use Case	Suitable for small or unsorted data.	Suitable for large and sorted data.

Conclusion:

- Linear search is simple but slow.
- Binary search is fast but works only on sorted data. ★

Q12) Sorting : Bubble Sort, Selection Sort, Insertion Sort

Introduction :

Sorting is the process of arranging elements in a particular order (ascending or descending). It helps in faster searching and better data management.

1. Bubble Sort

Definition : Bubble sort is a simple sorting technique in which adjacent elements are compared and swapped if they are in the wrong order.

Working :

- Compare two adjacent elements
- Swap if first is greater than second
- Repeat for all elements
- Largest element moves to the end in each pass

Example :

5 3 2 → 3 5 2 → 3 2 5 → 2 3 5

2. Selection Sort

Definition : Selection sort selects the smallest element from the array and places it at the correct position.

Working :

- Find the smallest element
- Swap it with the first element
- Repeat for remaining array

Example : 5 3 2 → 2 3 5

3. Insertion Sort

Definition : Insertion sort places each element at its correct position in the sorted part of the array.

Working :

- Take one element at a time
- Compare with previous elements
- Insert at correct position

Example : 5 3 2 → 3 5 2 → 2 3 5

Conclusion :

- Bubble sort is easy but less efficient.
- Selection sort does fewer swaps.

UNIT 2: Arrays

PAGE NO. : _____

DATE : ____/____/____

Q13) 2D Array Representation ★

Introduction :

A two-dimensional (2D) array is used to store data in the form of rows and columns, like a table or matrix.

Definition :

A 2D Array is an array of arrays where elements are stored in row and column format and each element is accessed using two indices.

Example :

$A = \begin{bmatrix} [10, 20, 30], \\ [40, 50, 60] \end{bmatrix}$

$A[0][0] = 10$
 $A[0][1] = 20$
 $A[1][2] = 60$

Columns $\longrightarrow$		
0	1	2
10	20	30
40	50	60

Explanation :

- First index represents **row**.
- Second index represents **column**.
- Elements are stored in a tabular form.

UNIT 2: Arrays

PAGE NO. : _____

DATE : ____/____/____

Q14) Advantages & Disadvantages of Array ★

Introduction :

An array is a widely used data structure for storing multiple elements of the same type. It has both advantages and limitations.

Advantages of Array :

① Easy Access

Elements can be accessed directly using index (fast access).

② Simple Implementation

Easy to understand and use in programs.

③ Efficient Storage

Stores multiple elements of the same type in a single structure.

④ Less Memory Overhead

Does not require extra memory for pointers.

Disadvantages of Array :

① Fixed Size

Size cannot be changed after declaration.

② Wastage of Memory

Extra space may remain unused.

③ Insertion & Deletion Difficult

Requires shifting of elements, which takes time.

④ Same Data Type Only

Cannot store different types of data.

Conclusion :

Thus, arrays are simple and efficient for storing data, but they have limitations like fixed size and difficulty in modification.

UNIT 3 : Linked List

PAGE NO. : _____

DATE : ____/____/____

Q) What is Linked List?

Introduction :

A linked list is an important data structure used for dynamic data storage. It overcomes some limitations of arrays.

Definition :

A Linked List is a collection of nodes where each node contains data and a link (pointer) to the next node in the sequence.

Representation :



Each node has two parts :

- Data → stores the value
- Next → stores address of next node

Explanation :

Elements are not stored in contiguous memory.

Nodes are connected using pointers.

The first node is called head.

The last node points to NULL.

Example :



Q16) Types of Linked List

Introduction :

Linked list can be classified into different types based on how nodes are connected with each other.

1. Singly Linked List

Definition :

A singly linked list is a type of linked list in which each node contains data and a link to the next node.

Explanation :

- Traversal is possible in one direction only.
- Last node points to NULL.



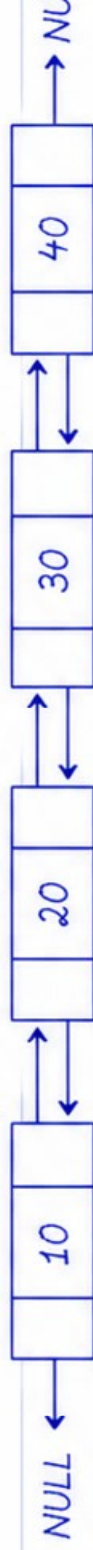
2. Doubly Linked List

Definition :

A doubly linked list is a type of linked list in which each node contains data, a link to the previous node, and a link to the next node.

Explanation :

- Traversal is possible in both directions.



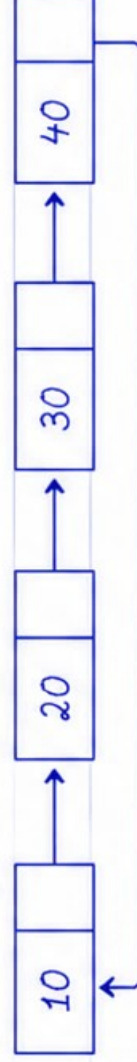
3. Circular Linked List

Definition :

A circular linked list is a type of linked list in which the last node points back to the first node, forming a circle.

Explanation :

- No node points to NULL.
- Forms a circular structure.



UNIT 3 : Linked List

PAGE NO. : _____

DATE : ____/____/____

Q17) Difference : Array vs Linked List ★

Introduction :

Array and linked list are two important data structures used to store data. They differ in memory storage, size, and operations.

Difference :

Basis	Array	Linked List
Definition	A collection of elements stored in contiguous memory locations	A collection of nodes connected using links (pointers)
Memory Allocation	Fixed memory allocation	Dynamic memory allocation
Size	Size is fixed	Size can grow or shrink
Access	Direct access using index (fast)	Sequential access (slower)
Insertion/Deletion	Difficult (requires shifting)	Easy (no shifting required)
Memory Usage	Less memory (no extra pointers)	More memory (extra pointer required)
Structure	Simple structure	Complex structure
Example	[10, 20, 30]	10 → 20 → 30 → NULL

Explanation (Extra lines) :

- Arrays store elements in continuous memory, so accessing elements is faster.
- Linked lists store elements in different locations connected by pointers, which makes insertion and deletion easier.

Q18) Insertion in Linked ListIntroduction :

Insertion is an operation used to add a new node into a linked list at a specific position.

Definition :

Insertion in Linked List means adding a new node containing data into the list by adjusting the links (pointers).

Types of Insertion :1. Insertion at Beginning

Before :



- Create a new node

After (Insert 5 at Beginning):



- Point new node to current head
- Update head to new node

2. Insertion at End

Before :



Steps :

- Traverse till last node
- Point last node to new node
- New node points to NULL

After (Insert 40 at End):

3. Insertion at Middle (Position)

Steps :

- Traverse to required position
- Adjust links of previous and next nodes
- Insert new node in between

Before :



After (Insert 25 between 20 and 30):

Example :

Before : 10 → 20 → 30

Insert 25 → 10 → 20 → 25 → 30

Q19) Deletion in Linked List

Introduction :

Deletion is an operation used to remove a node from a linked list.

Definition :

Deletion in Linked List means removing a node from the list by adjusting the links (pointers).

Types of Deletion :

1. Deletion at Beginning

Before :



Steps :

- Move head to the next node
- Delete the old first node

After (Delete 10 at Beginning) :



2. Deletion at End

Steps :

- Traverse to second last node
- Set its next to NULL
- Delete last node

Before :



After (Delete 30 at End) :



3. Deletion at Middle (Position)

Steps :

- Traverse to node before the one to delete
- Change its link to skip the node
- Delete the required node

Before :



After (Delete 20 from Middle) :



Example :

Before : 10 → 20 → 30

Delete 20 →

Conclusion :

Thus, deletion in a linked list is efficient as it only requires changing links without shifting elements.

Q20) Traversing Linked List

Introduction :

Traversing is a basic operation in a linked list used to access and display all elements.

Definition :

Traversing in Linked List means visiting each node of the list one by one from the first node to the last node.

Representation :



Algorithm (Steps) :

1. Start
2. Set pointer to head node
3. While pointer is not NULL :
 - Display data
 - Move pointer to next node
4. Stop

Example :

Linked List : 10 → 20 → 30 → 40 → NULL

Traversal → 10 → 20 → 30 → 40

Conclusion :

Thus, traversing helps in accessing all elements of a linked list sequentially.

Q21) Searching in Linked List

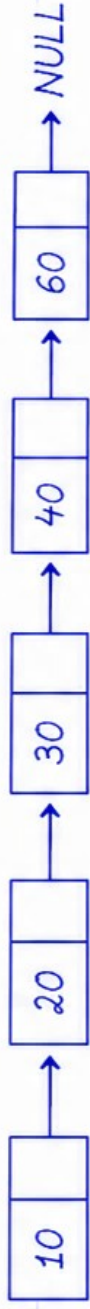
Introduction :

Searching is an operation used to find a specific element in a linked list.

Definition :

Searching in Linked List means finding a node with a given value by checking each node one by one.

Representation :



Algorithm (Steps) :

1. Start
2. Set pointer to head node
3. Compare each node's data with the search value
4. If match found $\rightarrow$ display position
5. Else move to next node
6. Repeat until found or reach NULL
7. If not found $\rightarrow$ display "Not Found"
8. Stop

Example :

- Linked List : $10 \rightarrow 20 \rightarrow 30 \rightarrow 40 \rightarrow \text{NULL}$
- Search = 30
- Check $\rightarrow 10 \times \rightarrow 20 \times \rightarrow 30 \checkmark$

Element found at position 2

Q22) Advantages & Disadvantages of Linked List

Introduction :

A linked list is a dynamic data structure that stores data using nodes and pointers. It has both advantages and disadvantages.

Advantages of Linked List :

1. Dynamic Size
 - Size can grow or shrink during execution.
2. Easy Insertion and Deletion
 - No need to shift elements, only links are changed.
3. Efficient Memory Use
 - Memory is allocated as needed (no wastage like fixed arrays).
4. Flexible Structure
 - Can easily expand and modify data.

Disadvantages of Linked List :

1. Extra Memory Usage
 - Requires additional memory for pointers.
2. No Direct Access
 - Cannot access elements directly (no indexing).
3. Slower Access
 - Traversal takes more time compared to arrays.
4. Complex Structure
 - More difficult to implement than arrays.

Conclusion :

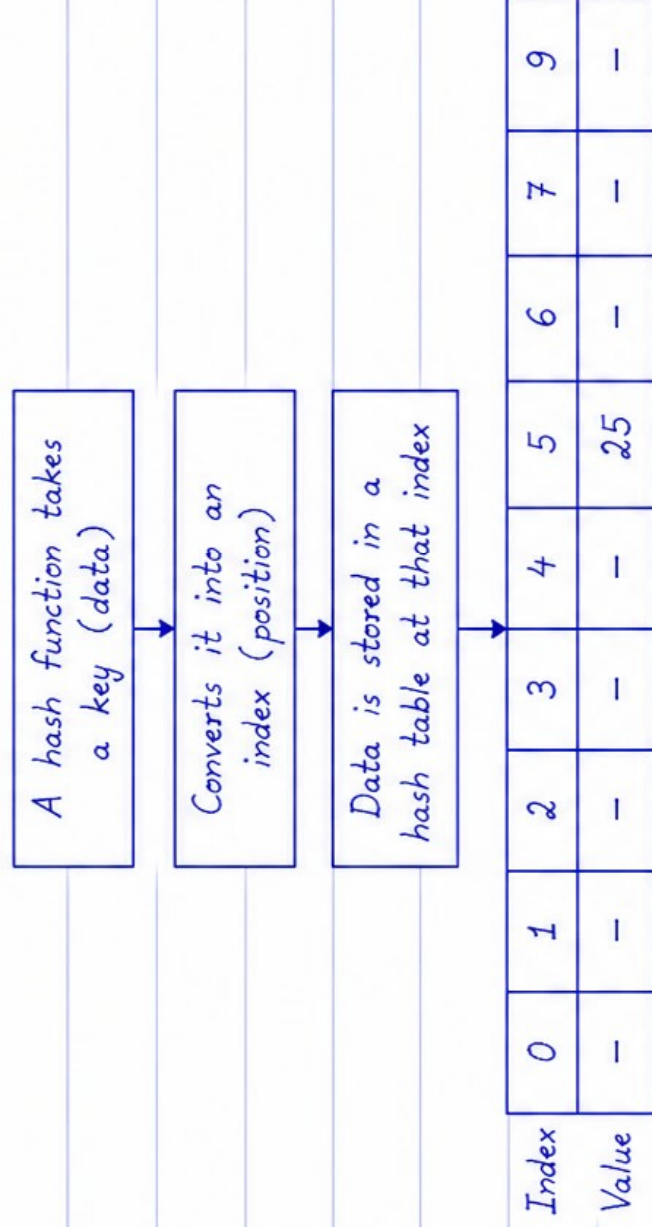
Thus, linked lists are flexible and efficient for insertion and deletion but have limitations like extra memory usage and

Q23) What is Hashing?Introduction :

Hashing is a technique used in data structures to store and retrieve data quickly.

Definition :

Hashing is a method of converting a key into a fixed index using a hash function, so that data can be stored and accessed efficiently.

Representation / Working :Example :

Key = 25

Hash Function: $h(k) = k \% 10$

$$h(25) = 25 \% 10 = 5$$

So, value is stored at index 5.

Explanation :

- Provides fast searching, insertion, and deletion
- Reduces time complexity compared to other methods

Q24) Hash Functions

Introduction :

A hash function is an important part of hashing. It is used to map data to a specific location in a hash table.

Definition :

A Hash Function is a function that takes a key as input and converts it into an index (address) in a hash table.

Types of Hash Functions :

1. Division Method

- $h(k) = k \% m$

(m = size of hash table)

2. Multiplication Method

- $h(k) = \lfloor m \times (k \times A \bmod 1) \rfloor$
($0 < A < 1$, m = size of hash table)

3. Mid-Square Method

- Square the key and take middle digits.

4. Folding Method

- Divide key into parts and add them.

Example :

Key = 27

$$h(k) = k \% 10$$

$$h(27) = 27 \% 10 = 7$$

So, value is stored at index 7.

Conclusion :

Thus, hash functions help in mapping keys to specific

positions for fast data access.

Q25) Collision Handling (Linear Probing, Chaining)

Introduction :

In hashing, sometimes two or more keys get the same index in the hash table. This situation is called a collision. To store all elements properly, we use collision handling techniques.

Definition :

Collision Handling is the process of resolving conflicts when multiple keys are mapped to the same index in a hash table.

1. Linear Probing (Open Addressing)

Introduction :

Linear probing is a collision handling technique used in hashing where, if a collision occurs, the algorithm searches for the next available empty position in the hash table sequentially.

Explanation :

- If a collision occurs, check the next position
- Continue checking sequentially until an empty slot is found
- It follows a simple linear sequence ($i, i+1, i+2, \dots$)

Example :

Hash table size = 10, $h(k) = k \% 10$

Insert keys: 23, 33, 43

Index	0	1	2	3	4	5	6	7	8	9
Value	-	23	-	33	43	-	-	-	-	-

$$h(23) = 3 \rightarrow \text{placed at 3}$$

$h(33) = 3 \rightarrow 3$ is full $\rightarrow$ next empty at 4 $\rightarrow$ placed at 4

$h(43) = 3 \rightarrow 3, 4$ full $\rightarrow$ next empty at 5 $\rightarrow$ placed at 5

2. Chaining (Separate Chaining)

Introduction :

Chaining is a collision handling technique in which each index of the hash table stores a linked list, and all elements that hash to the same index are stored in that list.

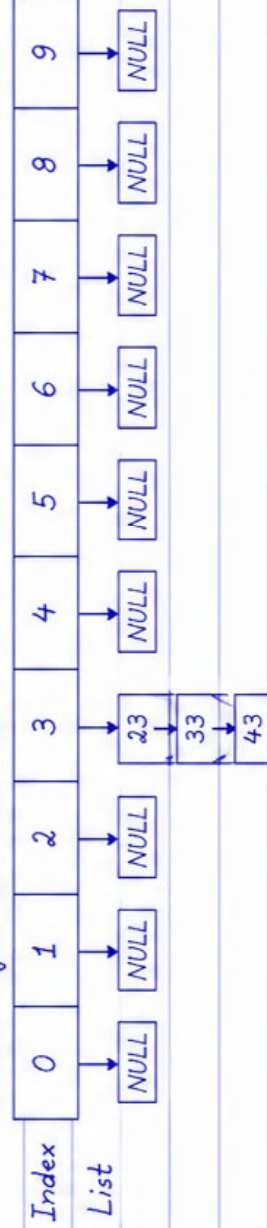
Explanation :

- Each index has a linked list
- Multiple elements can be stored at the same index
- New elements are added to the list

Example :

Hash table size = 10, $h(k) = k \% 10$

Insert keys: 23, 33, 43

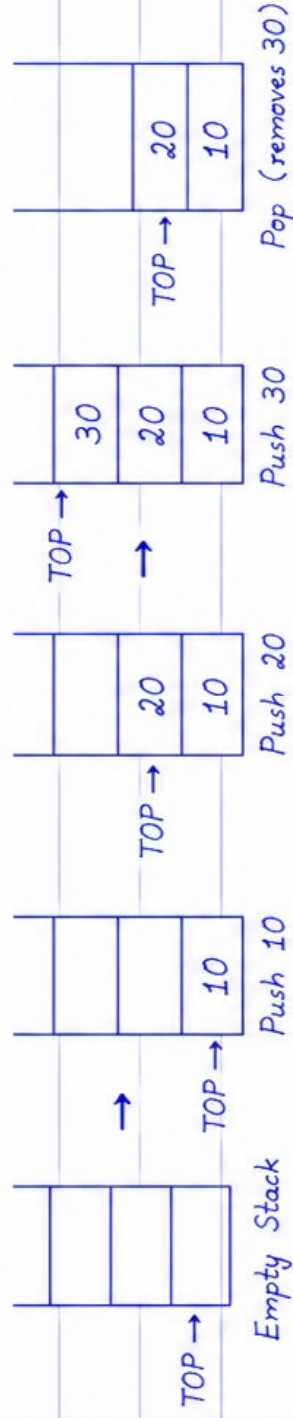


Q26) What is Stack?Introduction :

A stack is a linear data structure used to store elements in a particular order. It is commonly used in programming and data management.

Definition :

A Stack is a linear data structure in which elements are inserted and removed only from one end called the TOP, and it follows the principle of LIFO (Last In, First Out). This means the element which is inserted last is the first one to be removed, making it a restricted form of list where operations are allowed only at one end.

Representation :Basic Operations :

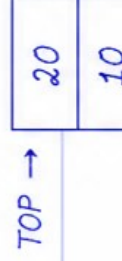
- Push → Insert element
- Pop → Remove element
- Peek → View top element
- IsEmpty → Check empty

Example :

Push → 10, 20, 30

Stack : TOP → 30 → 20 → 10

Pop → removes 30



Q27) Operations : PUSH & POPIntroduction :

Stack operations are used to insert and remove elements from the stack. The main operations are PUSH and POP, which follow the LIFO principle.

1. PUSH OperationDefinition (Long) :

PUSH is an operation used to insert a new element into the stack.

The element is always added at the top position, and the TOP pointer is updated accordingly. If the stack is full and no more elements can be added, it results in an overflow condition.

Explanation :

- Element is added only at the TOP
- TOP pointer moves upward
- Follows Last In, First Out (LIFO) rule

Example :

Stack : 10, 20 New Stack : TOP →

30	← TOP
20	←
10	←

Push 30

2. POP OperationDefinition (Long) :

POP is an operation used to remove an element from the stack. The element is removed from the top position, and the TOP pointer is updated. If the stack is empty and no element is available for removal, it results in an underflow condition.

Explanation :

- Element is removed from the TOP
- TOP pointer moves downward
- Follows LIFO principle

Example :

Stack : 10, 20, 30 After POP : TOP →

20	← TOP
10	←

Pop → removes 30

Q28) Stack using ArrayIntroduction :

A stack can be implemented using different data structures.

One of the simplest ways is using an array.

Definition (Long) :

A Stack using Array is an implementation of the stack data structure where elements are stored in a linear array, and operations like insertion and deletion are performed using an index called TOP, following the LIFO (Last In, First Out) principle.

Representation :

Index	0	1	2	3	4		0	1	2	3	4		0	1	2	3	4
A[]	-	-	-	-	-		10	-	-	-	-		10	20	-	-	-

$TOP = -1$ $TOP = 0$ $TOP = 1$

- Stack is represented using an array
- A variable TOP is used to track the top element
- Initially, $TOP = -1$ (stack is empty)

Explanation :

- When an element is inserted, TOP is increased and element is stored
- When an element is removed, TOP is decreased
- All operations happen at the TOP position only
- Stack size is fixed because array size is fixed

Example :

Array size = 5

Push 10 $\rightarrow TOP = 0$

Push 20 $\rightarrow TOP = 1$

Push 30 $\rightarrow TOP = 2$

Stack :

Index	0	1	2	3	4
A[]	10	20	30	-	-

$TOP = 2$

Q29) Stack using Linked List

Introduction :

A stack can also be implemented using a linked list.

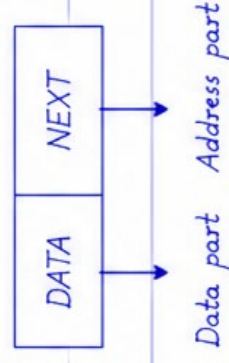
This method overcomes the limitation of fixed size in array implementation.

Definition (Long) :

A Stack using Linked List is an implementation of a stack where elements are stored as nodes in a linked list, and insertion and deletion are performed at one end called the TOP, following the LIFO (Last In, First Out) principle.

Representation :

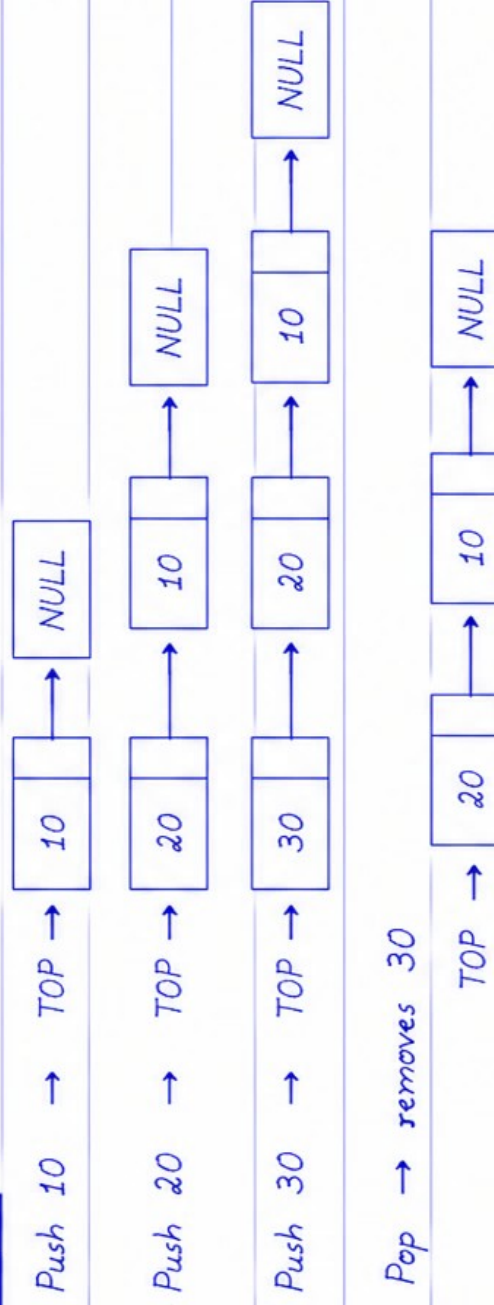
- Each node contains data + next pointer
- A pointer called TOP points to the first node
- The last node points to NULL



Explanation :

- Insertion (Push) is done at the beginning of the linked list
- Deletion (Pop) is also done from the beginning
- No need to shift elements
- Size is dynamic (can grow or shrink)

Example :



Q30) Applications of Stack

Introduction :

A stack is a very useful data structure that is widely used in computer science for solving many real-life and programming problems.

Definition (Long) :

The applications of stack refer to the various areas where the stack data structure is used to perform operations efficiently using the LIFO (Last In, First Out) principle.

Applications of Stack :

1. Expression Evaluation

- Used to evaluate infix, postfix, and prefix expressions
- Helps in solving mathematical calculations

2. Function Calls / Recursion

- Stack is used to store function calls in programs
- Helps in managing recursion

3. Undo and Redo Operations

- Used in applications like text editors
- Last action is undone first

4. Parenthesis Checking

- Used to check balanced parentheses in expressions
- Example: (), { }, []

5. Reversing Data

- Stack can be used to reverse strings or numbers

6. Backtracking

- Used in problems like maze solving, path finding
- Helps in going back to previous steps

Q31) Infix to Postfix Expression ★

Introduction:

Infix and postfix are different ways of writing mathematical expressions. Stack is used to convert infix expressions into postfix form.

Definition (Long):

Infix to Postfix conversion is the process of converting an expression written in infix form (operator between operands) into postfix form (operator after operands) using a stack, so that the expression can be evaluated easily by a computer.

Basic Idea:

- Infix $\rightarrow A + B$
- Postfix $\rightarrow AB +$
- Stack is used to store operators temporarily

Rules (Important):

- Operands (A, B , numbers) $\rightarrow$ directly add to output
- Operators ($+, -, *, /$) $\rightarrow$ push into stack
- Higher priority operator first ($*, / > +, -$)
- Parentheses are handled using stack

Representation / Working:

Example Expression (Infix) : $A + B * (C - D) / E$

Step	Scanned Symbol	Action	Stack (Top $\rightarrow$)	Output (Postfix)
1	A	Operand $\rightarrow$ Add to output	Empty	A
2	+	Push operator	+	A
3	B	Operand $\rightarrow$ Add to output	+	AB
4	*	Higher priority $\rightarrow$ Push	* +	AB
5	(	Push '('	(* +	AB
6	C	Operand $\rightarrow$ Add to output	(* +	ABC
7	-	Push operator	- (* +	ABC
8	D	Operand $\rightarrow$ Add to output	- (* +	ABCD
9	)	Pop until '('	* +	ABCD -
10	/	Higher priority than '+' $\rightarrow$ Push	/ * +	ABCD -
11	E	Operand $\rightarrow$ Add to output	/ * +	ABCD - E
12	End	Pop all operators	Empty	ABCD - E / * +

Postfix Expression : $ABCD - E / * +$

Precedence:

1. $*, / \rightarrow$ Highest
2. $+, -, \rightarrow$ Lower

Conclusion:

Stack helps in handling operators and parentheses to convert infix expressions into postfix form

Q32) What is Recursion?

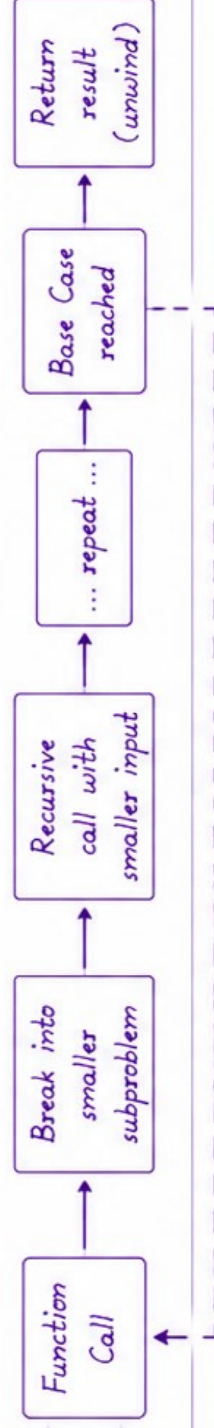
Introduction :

Recursion is an important concept in programming used to solve problems by breaking them into smaller parts. It is widely used in algorithms and data structures.

Definition (Long) :

Recursion is a programming technique in which a function calls itself repeatedly to solve a problem. It works by dividing a complex problem into smaller subproblems of the same type, until a simple condition called the base case is reached, where the recursion stops.

Representation / Working :



Explanation :

- A recursive function calls itself.
- Each call works on a smaller part of the problem.
- Must have a base case to stop recursion.
- Uses function call stack to store calls.

Example :

Factorial of 5

$$5! = 5 \times 4 \times 3 \times 2 \times 1$$

Using recursion:

$$\begin{aligned}
 \text{fact}(5) &= 5 \times \text{fact}(4) && // \text{ calls itself with } 4 \\
 &= 5 \times (4 \times \text{fact}(3)) && // \text{ calls itself with } 3 \\
 &= 5 \times (4 \times (3 \times \text{fact}(2))) && // \text{ calls itself with } 2 \\
 &= 5 \times (4 \times (3 \times \text{fact}(1))) && // \text{ calls itself with } 1 \\
 &= 5 \times (4 \times (3 \times (2 \times 1))) && // \text{ base case: } \text{fact}(1) = 1 \\
 &= 5 \times (4 \times (3 \times 2)) \\
 &= 5 \times (4 \times 6) \\
 &= 5 \times 24 \\
 &= 120
 \end{aligned}$$

Function calls
(go down)



Returning
results
(unwind)

Base Case :

$$\text{fact}(1) = 1 \quad (\text{stop recursion})$$

Q33) Advantages & Disadvantages of Recursion

Introduction :

Recursion is a powerful programming technique used to solve complex problems by dividing them into smaller subproblems. However, it has both advantages and disadvantages.

Advantages of Recursion :

① Simple and Easy to Understand

Recursive solutions are often shorter and easier to write compared to iterative methods.

② Reduces Code Length

Complex problems can be solved with fewer lines of code.

③ Useful for Complex Problems

Best suited for problems like factorial, Fibonacci, tree traversal, etc.

④ Better Problem Representation

Makes the logic of the problem clearer and more structured.

Disadvantages of Recursion :

① High Memory Usage

Each function call uses stack memory, which increases memory consumption.

② Slower Execution

Multiple function calls make recursion slower than iteration.

③ Risk of Infinite Recursion

If base case is missing, the program may run infinitely.

④ Stack Overflow

Too many recursive calls can cause stack overflow error.

Conclusion :

Thus, recursion is easy and powerful but should be used

carefully due to higher memory usage and performance issues.

Q34) Factorial using Recursion

Introduction :

Factorial is a common example used to explain recursion. It shows how a problem can be solved by calling the same function repeatedly.

Definition (Long) :

Factorial using recursion is a method of calculating the factorial of a number by using a recursive function that calls itself until it reaches a base case. The factorial of a number n is the product of all positive integers from 1 to n .

Mathematical Formula :

$$n! = n \times (n-1)!$$

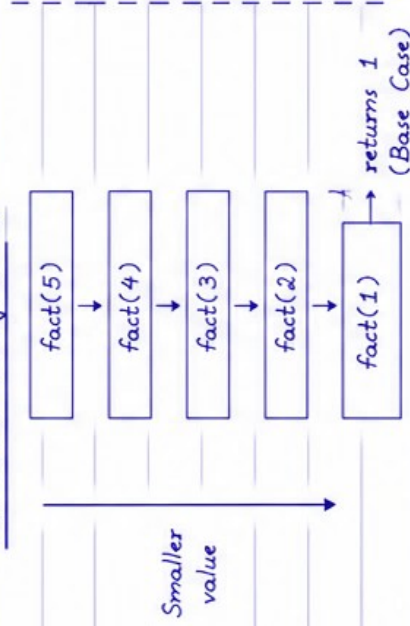
Base Case: $1! = 1$ or $0! = 1$

Representation / Working :

```
Recursive Function:
fact(n)
{
    if (n == 0 || n == 1) // Base case
        return 1;
    else
        return n * fact(n-1); // Recursive call
}
```

Step	Function Call	Value of n	Returns
1	fact(5)	5	$5 \times \text{fact}(4)$
2	fact(4)	4	$4 \times \text{fact}(3)$
3	fact(3)	3	$3 \times \text{fact}(2)$
4	fact(2)	2	$2 \times \text{fact}(1)$
5	fact(1)	1	1 (Base Case)

Call Stack (Going Down)



Returning Results (Going Up)



Example :

Find 5! using recursion.

$$\begin{aligned}
 \text{fact}(5) &= 5 \times \text{fact}(4) \\
 &= 5 \times (4 \times \text{fact}(3)) \\
 &= 5 \times (4 \times (3 \times \text{fact}(2))) \\
 &= 5 \times (4 \times (3 \times (2 \times \text{fact}(1)))) \\
 &= 5 \times (4 \times (3 \times (2 \times 1))) \quad [\text{fact}(1) = 1] \\
 &= 5 \times (4 \times (3 \times 2)) \\
 &= 5 \times (4 \times 6) \\
 &= 5 \times 24 \\
 &= 120
 \end{aligned}$$

$$\text{Therefore, } 5! = 120$$

Explanation :

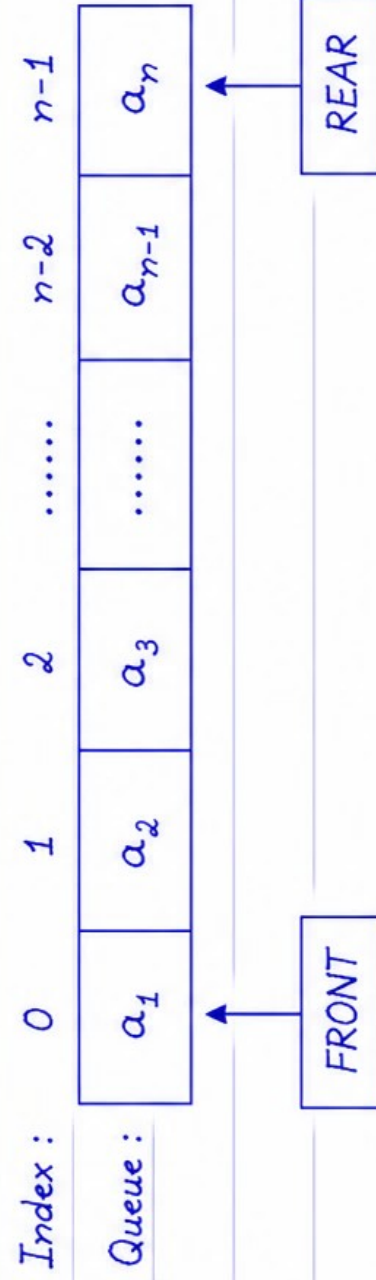
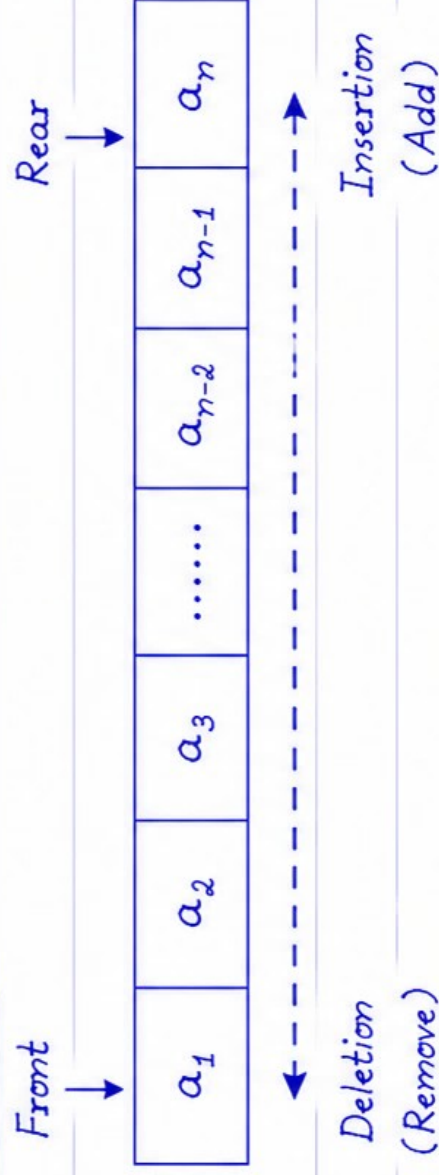
- Function calls itself with smaller value $(n-1)$.
- Continues until base case is reached.

Q35) What is Queue?Introduction :

A queue is a linear data structure used to store elements in a specific order. It is commonly used in real-life situations like waiting lines.

Definition (Long) :

A Queue is a linear data structure in which elements are inserted at one end called the rear and removed from the other end called the front, following the principle of FIFO (First In, First Out). This means the element that is inserted first is the first one to be removed.

Representation :

→ Front → where deletion happens.
→ Rear → where insertion happens.

Q36) Operations of Queue

Introduction :

Queue operations are used to insert, delete, and access elements in a queue. These operations follow the FIFO (First In, First Out) principle.

Definition (Long) :

The operations of a queue are the basic actions performed on the queue data structure to manage elements, such as inserting elements at the rear, removing elements from the front, and checking the status of the queue.

Main Operations of Queue :

1. Enqueue (Insertion)

Definition (Long) :

Enqueue is the operation used to insert a new element into the queue at the rear end. After insertion, the rear pointer moves forward. If the queue is full, it results in an overflow condition.

Representation :



Explanation :

- Element is added at the rear
- Rear pointer is updated
- Follows FIFO rule

Example :

Queue: 10 → 20
Enqueue 30

New Queue: 10 → 20 → 30

2. Dequeue (Deletion)

Definition (Long) :

Dequeue is the operation used to remove an element from the queue from the front end. After deletion, the front pointer moves forward. If the queue is empty, it results in an underflow condition.

Representation :



Explanation :

- Element is removed from the front
- Front pointer is updated

Example :

Queue: 10 → 20 → 30

Dequeue → removes 10

Q37) Types of Queue (Simple, Circular, Deque, Priority)

Introduction :

Queues are classified into different types based on how elements are inserted and deleted. Each type has its own working method and use.

Definition (Long) :

Types of Queue refer to the different variations of queue data structure designed to improve efficiency and overcome limitations of a simple queue.

1. Simple Queue

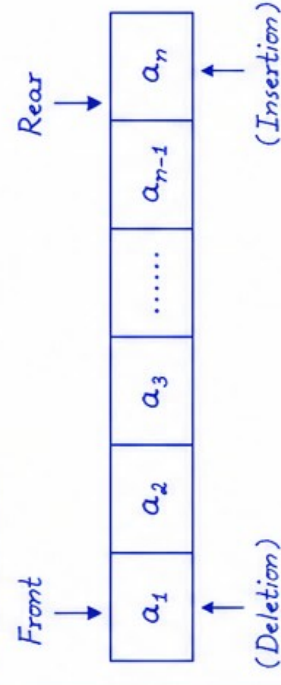
Definition :

A Simple Queue is a linear queue in which insertion is done at the rear and deletion is done from the front, following the FIFO principle.

Explanation :

- Basic form of queue
- May cause wastage of space

Representation :



2. Circular Queue

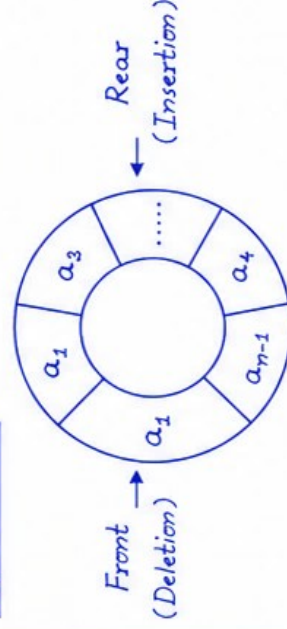
Definition :

A Circular Queue is a queue in which the last position is connected back to the first position to form a circle.

Explanation :

- Efficient use of memory
- Rear connects to front

Representation :



3. Deque (Double Ended Queue)

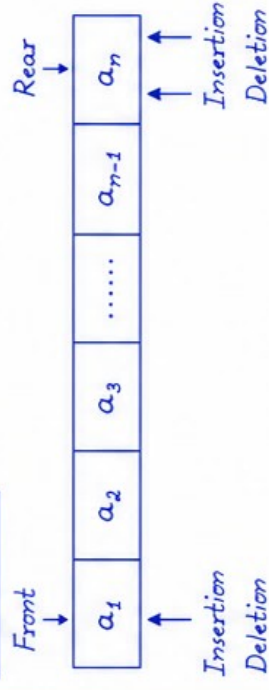
Definition :

A Deque is a type of queue in which insertion and deletion can be performed at both ends (front and rear).

Explanation :

- More flexible than simple queue
- Can work like both stack and queue

Representation :



4. Priority Queue

Definition :

A Priority Queue is a type of queue in which each element has a priority, and elements are processed based on priority rather than order.

Explanation :

- Higher priority element is served first

Representation (Example) :

Element	Priority	Order of Removal
A	3	(Highest Priority) ↑ (Lowest Priority)
B	1	
C	4	
D	2	

Q38) What is Graph?

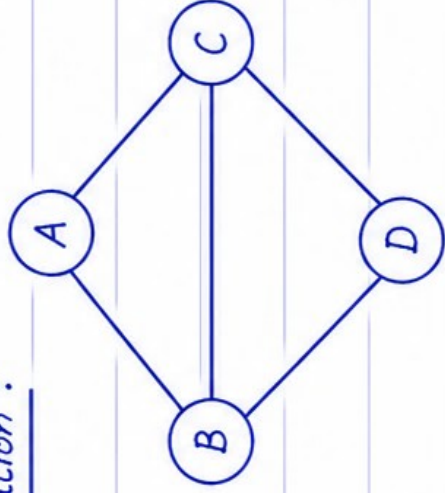
Introduction :

A graph is a non-linear data structure used to represent relationships between different elements. It is widely used in computer science for networking and connections.

Definition (Long) :

A Graph is a non-linear data structure that consists of a set of vertices (nodes) and a set of edges (connections) that connect pairs of vertices. It is used to represent relationships between objects, where each connection shows how two elements are related.

Representation :



- Vertices (Nodes) → represent elements
- Edges → represent connections between nodes

Explanation :

- A graph can be directed (edges have direction) or undirected.
- It can also be weighted (edges have values).
- Used to model real-life problems like networks, maps, etc.

Example :

Vertices: A, B, C, D
Edges : (A-B), (B-C), (C-D)

Q39) BFS (Breadth First Search)

Introduction :

Traversal is an important operation in graphs used to visit all vertices.

Breadth First Search (BFS) is one of the most commonly used traversal techniques.

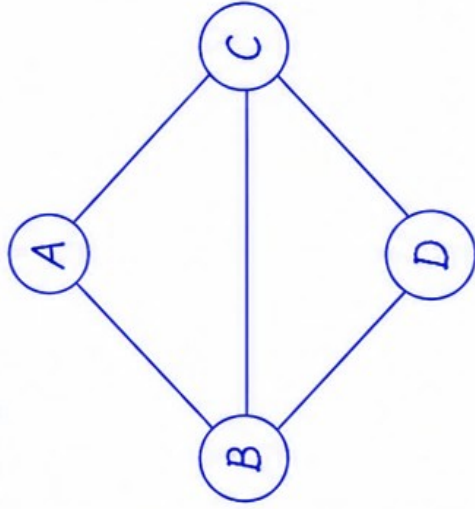
Definition (Long) :

Breadth First Search (BFS) is a graph traversal algorithm in which nodes are visited level by level, starting from a given source node.

It uses a queue data structure and explores all the neighbouring nodes first before moving to the next level.

Representation / Working :

Graph :



BFS Steps (Start from A) :

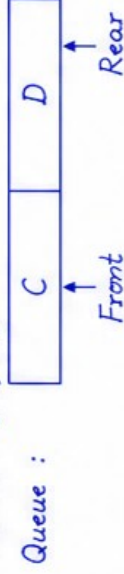
Step 1: Visit A. Enqueue its neighbours B, C.

Visited : {A}



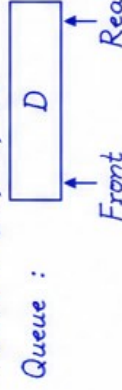
Step 2: Dequeue B. Visit B. Enqueue its neighbour D.

Visited : {A, B}



Step 3: Dequeue C. Visit C. (No new neighbour)

Visited : {A, B, C}



Step 4: Dequeue D. Visit D. (No new neighbour)

Visited : {A, B, C, D}



Explanation :

- Start from a source node.
- Visit all its adjacent (neighbour) nodes.
- Add neighbours to the queue.
- Remove nodes from queue one by one.
- Continue until all nodes are visited.

Example :

Graph nodes : A, B, C, D

Start from A

Traversal order :

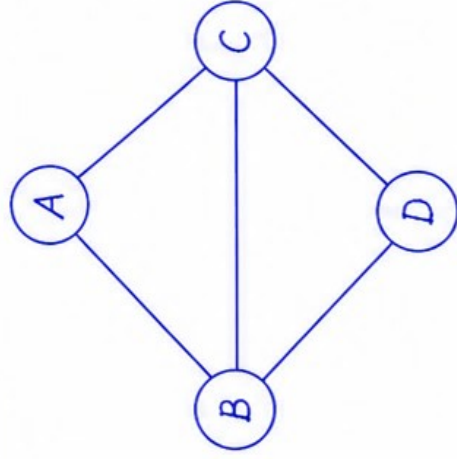


Q40) DFS (Depth First Search)Introduction :

Graph traversal is used to visit all vertices of a graph. Depth First Search (DFS) is another important traversal technique used in graphs.

Definition (Long) :

Depth First Search (DFS) is a graph traversal algorithm in which we start from a source node and explore as far as possible along each branch before backtracking. It uses a stack (or recursion) to keep track of nodes.

Representation / Working :Graph :Explanation :

- Start from a source node.
- Visit a node and go to its deepest neighbour.
- Continue until no further node is left.
- Then backtrack to previous node.
- Repeat until all nodes are visited.

Example :

Graph nodes : A, B, C, D

Start from A

Traversal order (one possible):

A → B → D → C

DFS Steps (Start from A) :

Step 1 : Visit A. Push A to stack.

Visited : {A}

Stack :

--

 A ← Top

Step 2 : From A go to its deepest neighbour B.

Push B to stack.

Visited : {A, B}

Stack :

--	--

 A B ← Top

Step 3 : From B go to its deepest unvisited neighbour D.

Push D to stack.

Visited : {A, B, D}

Stack :

--	--	--

 A B D ← Top

Step 4 : From D, no unvisited neighbour.

Pop D (backtrack to B).

Visited : {A, B, D}

Stack :

--	--

 A B ← Top

Step 5 : From B, no unvisited neighbour.

Pop B (backtrack to A).

Visited : {A, B, D}

Stack :

--

 A ← Top

Step 6 : From A, next unvisited neighbour is C.

Push C to stack.

Visited : {A, B, D, C}

Stack :

--	--

 A C ← Top

Step 7 : From C, no unvisited neighbour.

Pop C (backtrack to A).

Visited : {A, B, D, C}

Stack :

--

 A ← Top

Step 8 : From A, no more unvisited neighbour.

Pop A. Stack becomes empty. Traversal complete.

Visited : {A, B, D, C}

Stack :

--

 Empty

Q41) Representation of Graph

Introduction :

A graph can be represented in different ways in a computer so that it can be stored and processed efficiently.

Definition (Long) :

Graph Representation is the method of storing a graph in computer memory using suitable data structures, so that operations like traversal, searching, and path finding can be performed efficiently.

Types of Graph Representation :

1.

Adjacency Matrix

Definition :

An Adjacency Matrix is a 2D array where rows and columns represent vertices, and values indicate whether an edge exists.

Explanation :

- If edge exists $\rightarrow 1$
- If no edge $\rightarrow 0$
- Matrix size = $n \times n$
(n = number of vertices)

Example :

Consider a graph with vertices A, B, C, D and edges : $(A-B), (A-C), (B-D), (C-D)$

	A	B	C	D
A	0	1	1	0
B	1	0	0	1
C	1	0	0	1
D	0	1	1	0

If A is connected to $B \rightarrow$
 $\text{matrix}[A][B] = 1$

2.

Adjacency List

Definition :

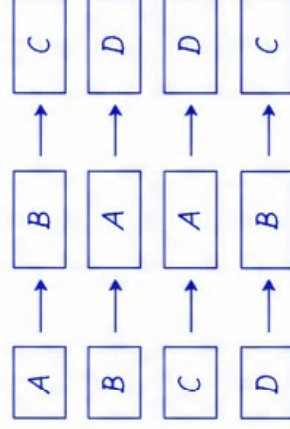
An Adjacency List represents a graph as a collection of lists, where each vertex stores a list of its adjacent vertices.

Explanation :

- Each node has a list of neighbours
- Uses linked list or array of lists
- Saves memory compared to matrix

Example :

Consider the same graph with vertices A, B, C, D and edges : $(A-B), (A-C), (B-D), (C-D)$



$A \rightarrow B, C$
 $B \rightarrow A, D$
 $C \rightarrow A, D$
 $D \rightarrow B, C$

Difference (Short for extra marks) :

Aspect	Adjacency Matrix	Adjacency List
Data Structure	Uses 2D array	Uses lists
Space Complexity	More memory ($O(n^2)$)	Less memory ($O(V + E)$)
Implementation	Easy to implement	More efficient
Edge Check	$O(1)$	$O(\text{degree of vertex})$
Best for	Dense graphs	Sparse graphs

Conclusion :

Thus, graphs can be represented using adjacency matrix and adjacency list depending on memory and efficiency requirements.

Q42) What is Tree?

PAGE NO. : _____

DATE : ____/____/____

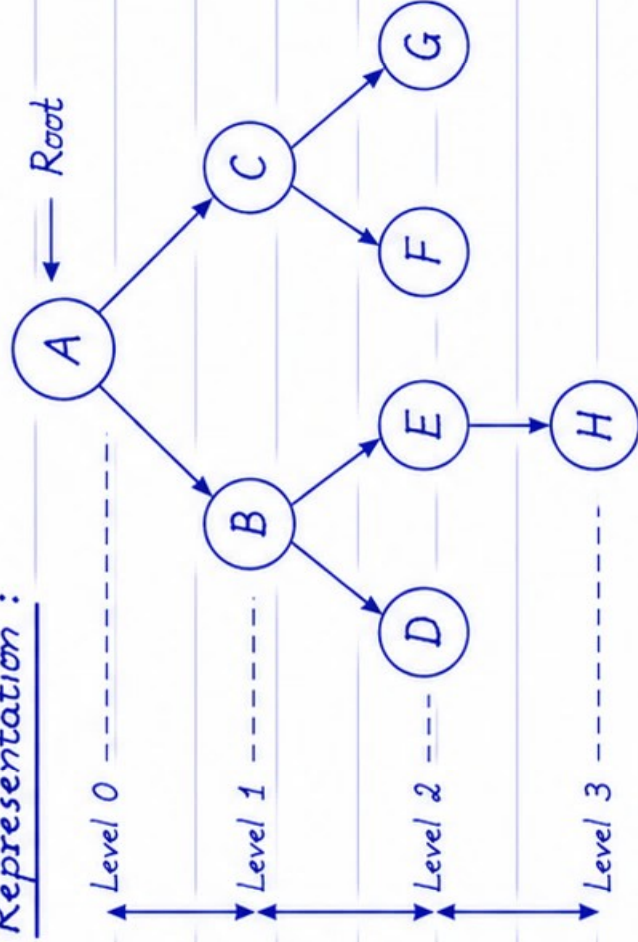
Introduction :

A tree is a non-linear data structure used to represent hierarchical relationships between elements. It is widely used in computer science for organizing data.

Definition (Long) :

A Tree is a non-linear data structure that consists of a set of nodes connected by edges in a hierarchical manner. It has a special node called the root, and every node can have zero or more child nodes. It does not contain any cycles and follows a parent-child relationship.

Representation :



- A is the Root node.
- B and C are children of A.
- D and E are children of B.
- F and G are children of C.
- H is the child of E.
- D, E, F, G are leaf nodes (having no children).
- Nodes can have zero or more children.

Key Points :

- Tree is a hierarchical data structure.
- It has exactly one root node.
- Every node (except root) has exactly one parent.
- A node can have zero or more children.
- It does not contain any cycles.

Q43) Binary Tree

PAGE NO. : _____

DATE : ____/____/____

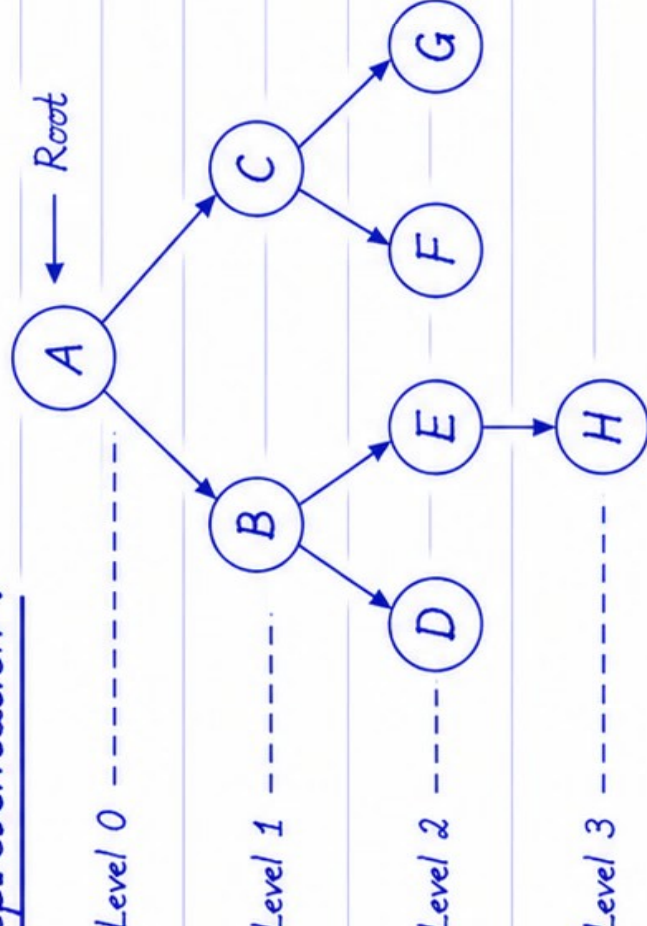
Introduction :

A binary tree is a special type of tree data structure that is widely used in computer science for efficient data organization and searching.

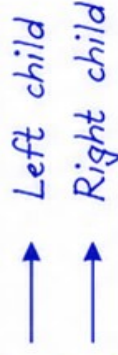
Definition (Long) :

A Binary Tree is a type of tree data structure in which each node can have at most two children, called the left child and the right child. It is used to represent hierarchical data in a structured and efficient manner.

Representation :



- A is the root node.
- B and C are children of A.
- D and E are children of B.
- F and G are children of C.
- H is the child of E.
- Each node has at most two children (left and right).



Key Points :

- It is a tree in which each node has at most two children.
- The two children are referred to as left child and right child.
- It follows a hierarchical structure.
- The order of children matters (left and right are different).
- It is widely used in searching, sorting, expression trees, heaps, etc.

4) Tree Traversal (Inorder, Preorder, Postorder)

Introduction :

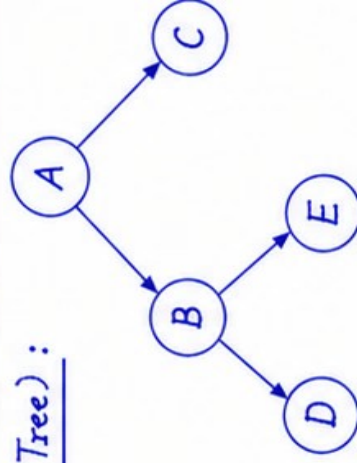
Tree traversal is the process of visiting all nodes of a tree in a specific order.

It is used to access and process data stored in a tree.

Definition (Long) :

Tree Traversal is a technique of visiting each node of a tree exactly once in a systematic way. In binary trees, there are three main types of traversal: Inorder, Preorder, and Postorder.

Representation (Example Tree) :



- A is the root node.
- B and C are children of A.
- D and E are children of B.
- C has no children.
- D and E are leaf nodes.

Inorder Traversal (LNR)

Definition :

Inorder Traversal, nodes are visited in the order :

Left → Node → Right

Explanation :

- First visit left subtree
- Then visit root
- Then visit right subtree

Example Output :

D → B → E → A → C

by step (Inorder) :

- Left of A (subtree B) → D, B, E
 - Visit root → A
 - Right of A (subtree C) → C
- Overall : D → B → E → A → C

Conclusion :

Thus, tree traversal methods are used to visit all nodes in different orders depending on the requirement.

2. Preorder Traversal (NLR)

Definition :

In Preorder Traversal, nodes are visited in the order :

Node → Left → Right

Explanation :

- First visit root
- Then left subtree
- Then right subtree

Example Output :

A → B → D → E → C

Step by step (Preorder) :

- Visit root → A
 - Left of A (subtree B) → B, D, E
 - Right of A (subtree C) → C
- Overall : A → B → D → E → C

3. Postorder Traversal (LRN)

Definition :

In Postorder Traversal, nodes are visited in the order :

Left → Right → Node

Explanation :

- First visit left subtree
- Then right subtree
- Then root

Example Output :

D → E → B → C → A

Step by step (Postorder) :

- Left of A (subtree B) → D, E
 - Right of A (subtree C) → C
 - Visit root → A
- Overall : D → E → B → C → A

Q45) Binary Search Tree (BST)

PAGE NO. : _____

DATE : ____/____/____

Introduction :

A Binary Search Tree (BST) is a special type of binary tree used for efficient searching, insertion, and deletion operations.

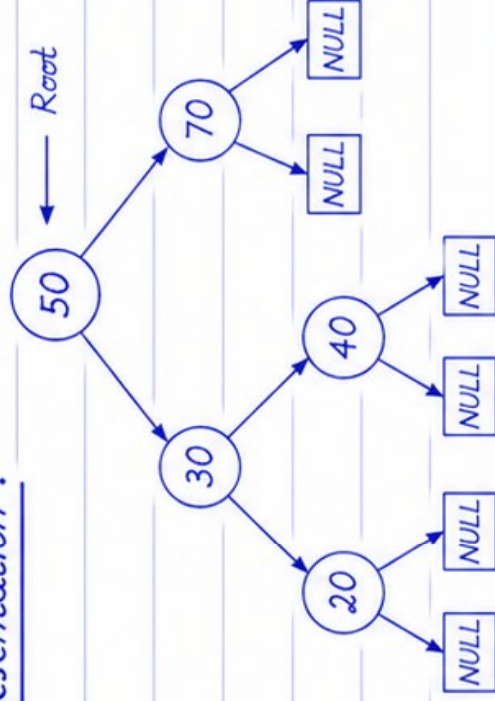
Definition (Long) :

A Binary Search Tree (BST) is a binary tree in which each node follows a specific property:

- All elements in the left subtree are less than the node.
- All elements in the right subtree are greater than the node.

This property is called the Binary Search Property and helps in fast searching.

presentation :



Explanation :

- Each node has at most two children.
- Left child contains smaller values.
- Right child contains larger values.
- This structure makes searching faster.

Example :

Insert values: 50, 30, 70, 20, 40

Step by Step Insertion :

Insert 50 → Becomes root.

Insert 30 → $30 < 50$, so insert in left of 50.

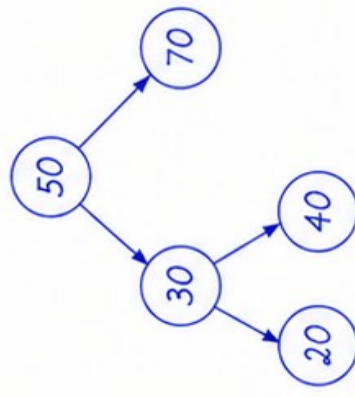
Insert 70 → $70 > 50$, so insert in right of 50.

Insert 20 → $20 < 50$, go left to 30; $20 < 30$, insert in left of 30.

Insert 40 → $40 < 50$, go left to 30; $40 > 30$, insert in right of 30.

Tree becomes :

- 50 → root
- 30 → left of 50
- 70 → right of 50
- 20 → left of 30
- 40 → right of 30



Operations in BST (Short) :

1. Insertion :

- Insert a new node in the appropriate position by following BST property.

2. Deletion :

- Delete a node and maintain BST property by adjusting the tree.

3. Searching :

- Search for a value by comparing with nodes and moving left or right.

Key Point :

BST follows the Binary Search Property which allows efficient searching, insertion and deletion.

Q46) Insertion & Deletion in BST

Introduction :

Binary Search Tree (BST) supports efficient insertion and deletion operations based on its ordering property ($\text{Left} < \text{Root} < \text{Right}$).

Definition (Long) :

Insertion and Deletion in BST are operations used to add and remove nodes in a binary search tree while maintaining the BST property, where all elements in the left subtree are smaller and in the right subtree are larger than the root.

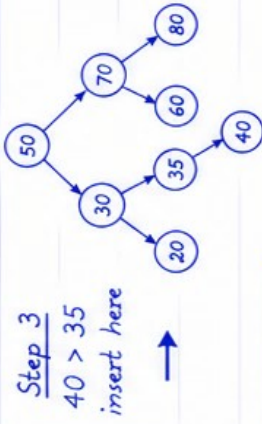
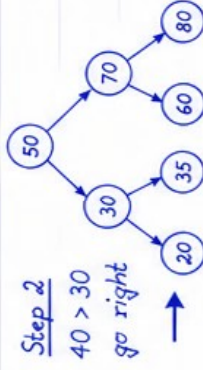
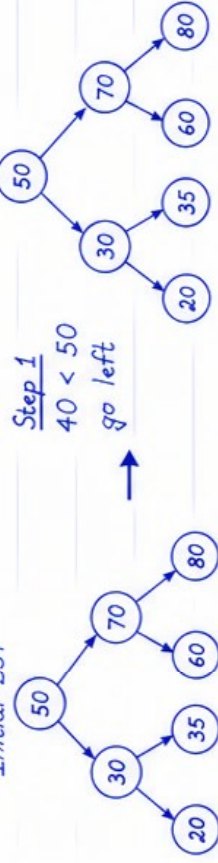
1. Insertion in BST

Explanation :

- Start from the root node.
- Compare new value with current node.
- If smaller $\rightarrow$ go to left subtree.
- If greater $\rightarrow$ go to right subtree.
- Continue until correct position is found.
- Insert the node at leaf position.

Example : Insert 40 in BST

Initial BST



40 is inserted at the correct position.

2. Deletion in BST

Explanation :

Deletion has 3 cases :

Case 1 : Deleting a Leaf Node

Simply remove the node.

Case 2 : Node with One Child

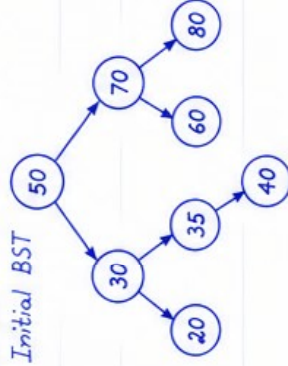
Replace node with its child.

Case 3 : Node with Two Children

Replace node with inorder successor (smallest in right subtree).
Then delete that successor node.

Example : Delete 30 from BST

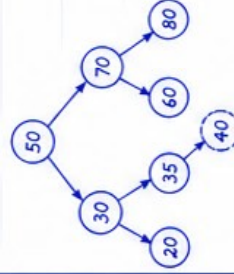
Initial BST



Node to delete = 30
30 has two children.
Inorder successor of 30
= smallest in right subtree
of 30 = 40

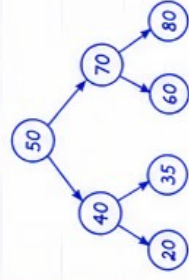
Step 1 :

Inorder successor
of 30 is 40.



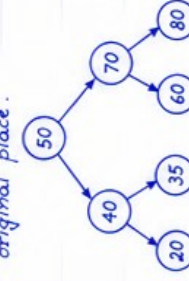
Step 2 :

Replace 30
with 40.



Step 3 :

Delete the
successor (40)
from its
original place.



30 is deleted and tree is adjusted correctly.

Conclusion :

Thus, insertion and deletion in BST maintain the binary search property

47) Difference between Stack and Queue

Conclusion :

Stack and queue are both linear data structures, but they differ the way elements are inserted and removed.

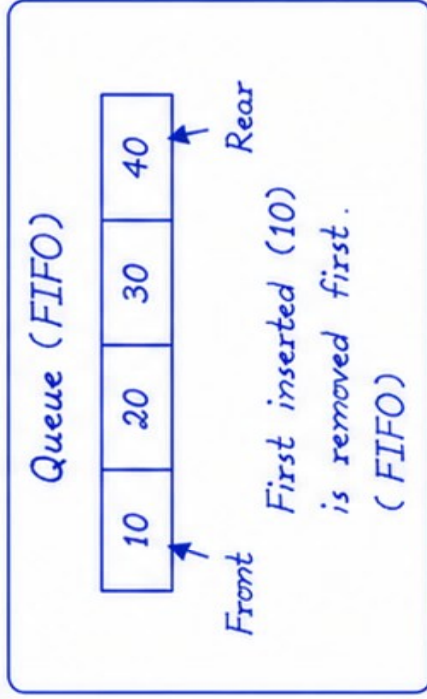
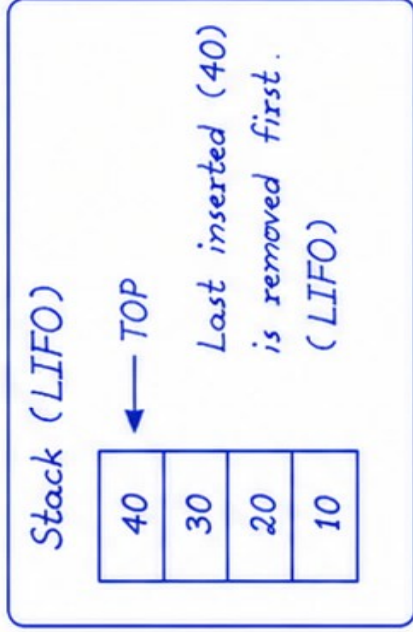
Conclusion :

Basis	Stack	Queue
Definition	A linear data structure that follows LIFO (Last In, First Out)	A linear data structure that follows FIFO (First In, First Out)
Order of Operation	Last inserted element is removed first.	First inserted element is removed first.
Operations Used	Insertion and deletion at one end (TOP)	Insertion at rear and deletion from front.
Operations	Push, Pop	Enqueue, Dequeue
Pointers Used	Only TOP pointer	Two pointers: Front and Rear
Example	Stack of plates	Queue (line of people)
Working Style	Works like a stack/pile	Works like a queue/line

Conclusion (Extra lines) :

In a stack, elements are added and removed from the same end, making it LIFO (Last In, First Out).

In a queue, elements are added at one end and removed from the other, making it FIFO (First In, First Out)



Introduction :

A linked list is a flexible data structure used to store data dynamically. It is widely used in many applications where frequent insertion and deletion are required.

Definition (Long) :

The applications of linked list refer to the various areas where linked lists are used to manage and organize data efficiently using dynamic memory and pointers.

Applications of Linked List :

No.	Application	Explanation with Example	
1.	Dynamic Memory Management	- Linked list allows dynamic allocation of memory. - Memory is used only when needed.	
2.	Implementation of Stack and Queue	- Linked list is used to implement stack and queue. - Helps in efficient insertion and deletion.	Stack (using Linked List) Queue (using Linked List)
3.	Polynomial Representation	- Used to represent polynomial equations. - Each term is stored as a node.	
4.	Graph Representation	- Used in adjacency list to represent graphs. - Efficient for storing connected data.	A → B → C → NULL B → D → NULL C → D → NULL D → NULL
5.	File System / Directory Structure	- Used to store and manage files in systems. - Helps in linking files and folders.	
6.	Undo / Redo Operations	- Used in applications like text editors. - Stores previous states.	

Conclusion :

Thus, linked list is used in many applications because of its dynamic memory

Q250) Explain Various Applications of Recursion

PAGE NO. : _____

DATE : ____/____/____

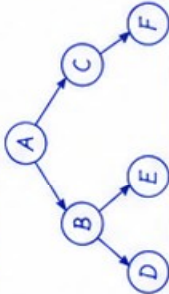
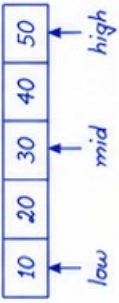
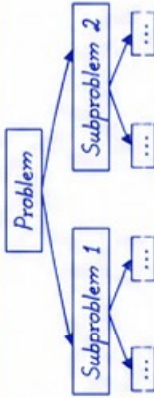
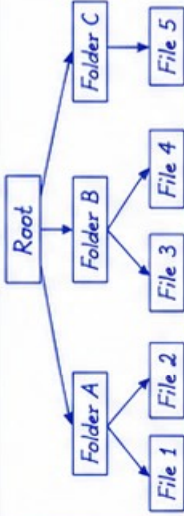
Introduction :

Recursion is a powerful technique used in programming to solve problems by breaking them into smaller subproblems. It is widely used in many areas of computer science.

Definition (Lang) :

The applications of recursion refer to the different areas where recursive functions are used to solve complex problems by repeatedly calling themselves until a base condition is reached.

Applications of Recursion :

No.	Application	Explanation	Example / Illustration
1.	Mathematical Calculations	- Used to calculate factorial, Fibonacci series, power of numbers. - Simplifies mathematical problems.	- Factorial: $n! = n \times (n-1)!$ - Fibonacci: $F(n) = F(n-1) + F(n-2)$ - Power: $a^n = a \times a^{(n-1)}$ $5! = 5 \times 4!$ $F(5) = F(4) + F(3)$ $2^5 = 2 \times 2^4$
2.	Tree Traversal	- Used to traverse trees (Inorder, Preorder, Postorder). - Each subtree is processed recursively.	Example Tree :  - Inorder (L N R) : D, B, E, A, C, F - Preorder (N L R) : A, B, D, E, C, F - Postorder (L R N) : D, E, B, F, C, A
3.	Searching and Sorting Algorithms	- Used in algorithms like binary search, quick sort, merge sort. - Helps in efficient problem solving.	- Binary Search (Recursive) - Quick Sort (Divide and Conquer) - Merge Sort (Divide and Conquer) 
4.	Backtracking Problems	- Used in problems like maze solving, N-Queens, puzzles. - Helps to explore all possible solutions.	Examples : - N-Queens Problem - Maze Solving - Sudoku Solver - Hamiltonian Path  (N-Queens Solution)
5.	Divide and Conquer	- Breaks problem into smaller parts. - Example: merge sort, quick sort.	Solve subproblems recursively and combine results. 
6.	File System Traversal	- Used to access files and folders in hierarchical structure. - Each folder can contain subfolders.	Traversal is done recursively for each subfolder. 
7.	Tower of Hanoi	- Classic example solved using recursion. - Demonstrates recursive logic clearly.	Move n disks from source (A) to destination (C) using auxiliary (B). 

Conclusion :

Recursion is used in a wide range of applications. It makes the code shorter, simpler and easier to understand for problems that can be broken into similar subproblems.

★ Key Point ★

Recursion solves problems by solving smaller versions of the same problem.

Introduction :

Graphs are widely used to represent relationships and connections between different entities. They are very useful in real-world applications like social networks and web crawling.

Definition (Long) :

The role of graphs refers to how graph data structures are used to model and solve real-world problems by representing objects as vertices (nodes) and their relationships as edges (connections).

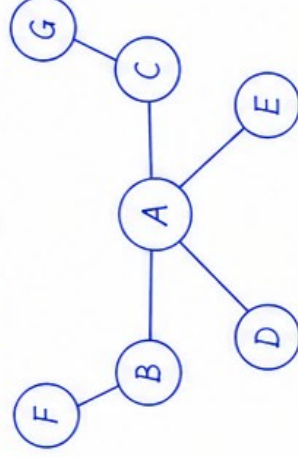
1. Role of Graphs in Social Networks

Explanation :

- Each user is represented as a node (vertex).
- Connections (friends/followers) are represented as edges.
- Helps in finding mutual friends, suggestions, and communities.
- Used in platforms like social media apps.

Example :

User A connected to B and C → shows friendship network.



Node
= User
Edge
= Friendship/
Connection

This graph helps in friend suggestions, mutual friends, community detection, etc.

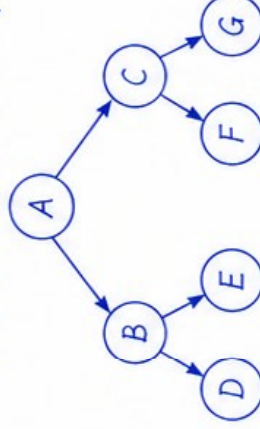
2. Role of Graphs in Web Crawling

Explanation :

- Web pages are represented as nodes.
- Links between pages are edges.
- Crawlers use graph traversal (BFS/DFS) to visit pages.
- Helps search engines index websites efficiently.

Example :

Page A links to B and C → crawler visits all linked pages.



Node
= Web Page
Edge
= Hyperlink

Crawler starts from page A and visits B, C, D, E, F, G using BFS / DFS.

Conclusion :

Thus, graphs play an important role in representing and solving problems in social networks and web crawling by modeling relationships and connections.

and Representation of Two-Dimensional Arrays in Memory

Introduction :

Arrays can store multiple values, but sometimes data is required in tabular form (rows and columns). For this purpose, multi-dimensional arrays are used.

Definition (Long) :

A Multi-Dimensional Array is an array that contains more than one dimension, meaning it stores data in the form of rows, columns, or multiple levels. It is basically an array of arrays and is used to represent complex data structures like matrices.

Concept of Multi-Dimensional Arrays :

- It can be 2D, 3D, or higher dimensions.
- Each element is accessed using multiple indices.
- Example :

Dimension	Representation
1D Array	$A[5]$
2D Array	$A[2][3]$
3D Array	$A[2][3][4]$

Two-Dimensional Array (2D Array)

Definition :

A 2D Array is a type of multi-dimensional array where data is stored in the form of rows and columns.

Representation :

- Represented as : $A[\text{row}][\text{column}]$
- Example : $A[2][3] \rightarrow 2 \text{ rows, } 3 \text{ columns}$

Representation in Memory :

There are two ways to store a 2D array in memory :

1. Row Major Order

- Elements are stored row by row.
- First complete row is stored, then next row.

Example :

$$A = \begin{array}{|c|c|c|} \hline 0 & 1 & 2 \\ \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline \end{array}$$

Memory :

1	2	3	4	5	6
0	1	2	3	4	5

Row Major Order

Address Mapping Formula (Row Major) :

$$\text{Address}(A[i][j]) = \text{Base} + [(i \times N) + j] \times w$$

Where,

- i = row index (0 to $M-1$)
- j = column index (0 to $N-1$)
- M = total number of rows
- N = total number of columns
- w = size of each element (in bytes)

2. Column Major Order

- Elements are stored column by column.
- First complete column is stored, then next column.

Example :

$$A = \begin{array}{|c|c|c|} \hline 0 & 1 & 2 \\ \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline \end{array}$$

Memory :

1	4	2	5	3	6
0	1	2	3	4	5

Column Major Order

Address Mapping Formula (Column Major) :

$$\text{Address}(A[i][j]) = \text{Base} + [(j \times M) + i] \times w$$

Where,

- i = row index (0 to $M-1$)
- j = column index (0 to $N-1$)
- M = total number of rows
- N = total number of columns
- w = size of each element (in bytes)

Explanation :

- Although array looks like a table, memory is linear.
- Mapping formula is used to calculate address of any element.
- Row-major order is more commonly used in languages like C.

2D Array (Table Form)

0	1	2
0	1	2
1	4	5
1	4	5

Memory (Linear Form)

(Row Major Example)

1	2	3	4	5	6
0	1	2	3	4	5

Conclusion :

Thus, multi-dimensional arrays help in storing complex data, and 2D arrays

and Their Advantages

Introduction :

In many applications, matrices contain a large number of zero values. Storing all elements wastes memory, so sparse matrices are used.

Definition (Long) :

A Sparse Matrix is a matrix in which most of the elements are zero and only a few elements are non-zero. Instead of storing all elements, only the non-zero elements and their positions are stored to save memory.

Concept of Sparse Matrix :

- Contains more zero values than non-zero values.
- Efficient storage technique.
- Used when data is mostly empty.

Representation (Triplet Form) :

Instead of storing full matrix, we store :

- Row number
- Column number
- Value

Triplet Form Structure

Row No.	Column No.	Value
(i)	(j)	(Value)

General Format :

- (i, j, value)

Where,

i = row index

j = column index

value = non-zero element

Example :

Matrix :

$$\begin{bmatrix} 0 & 0 & 5 \\ 0 & 0 & 0 \\ 8 & 0 & 0 \end{bmatrix}$$

Non-zero elements are :

$$(0, 2) = 5$$

$$(2, 0) = 8$$

Stored as (Triplet Form) :

Row	Column	Value
0	2	5
2	0	8

Advantages of Sparse Matrix :

No.	Advantage	Explanation
1.	Memory Efficient	• Stores only non-zero elements and their positions. • Saves a large amount of memory space.
2.	Faster Processing	• Since only non-zero elements are stored, less data is processed, making operations faster.
3.	Reduced Storage Cost	• Requires less storage space. • Lower cost for storage devices.
4.	Efficient for Large Data	• Very useful in large matrices where most of the elements are zero. • Example: Scientific computations, network analysis, graph representations.

Conclusion :

Thus, sparse matrices provide an efficient way to store and process data

Introduction :

In hashing, sometimes different keys get mapped to the same index in a hash table. This situation is called a collision, and it must be resolved to store data properly.

Definition (Long) :

Collision Resolution is the technique used to handle situations where multiple keys are assigned the same index in a hash table. It ensures that all elements are stored and accessed efficiently without data loss.

1. Open Addressing

Concept :

Open addressing is a method in which, if a collision occurs, we search for another empty position within the same hash table.

Explanation :

- All elements are stored inside the hash table.
- If one index is full $\rightarrow$ move to next available slot.
- Common techniques :
 - ① Linear Probing (check next slot)
 - ② Quadratic Probing
 - ③ Double Hashing

Example :

Let, $h(k) = k \% 10$

- Insert $25 \rightarrow h(25) = 25 \% 10 = 5 \rightarrow$ placed at index 5
- Insert $35 \rightarrow h(35) = 35 \% 10 = 5 \rightarrow$ collision!

Using Linear Probing $\rightarrow$ check next slot

Index 5 is full $\rightarrow$ move to index 6 $\rightarrow$ placed there.

Hash Table (Size = 10)

Index	0	1	2	3	4	5	6	7	8	9
Value	—	—	—	—	—	25	35	—	—	—

Note : Open addressing stores all elements in the same table by finding alternate empty slots.

2. Chaining (Separate Chaining)

Concept :

Chaining is a method where each index of the hash table contains a linked list, and all elements that hash to the same index are stored in that list.

Explanation :

- Each slot stores multiple elements using a list.
- No need to search for empty slot.
- Easy to implement.

Example :

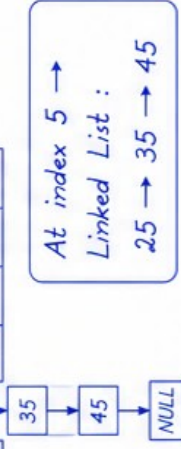
Let, $h(k) = k \% 10$

- Insert $25 \rightarrow h(25) = 5$
- Insert $35 \rightarrow h(35) = 5$ (collision)
- Insert $45 \rightarrow h(45) = 5$ (collision)

All elements stored in linked list at index 5.

Hash Table (Size = 10)

Index	0	1	2	3	4	5	6	7	8	9
List	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓
	NULL	NULL	NULL	NULL	NULL	25	NULL	NULL	NULL	NULL



At index 5 $\rightarrow$
Linked List :
 $25 \rightarrow 35 \rightarrow 45$

Note : Chaining allows multiple elements at the same index using linked list.
No need to find empty slot.

Comparison (Summary)

Basis	Open Addressing	Chaining (Separate Chaining)
Storage	All elements stored in the same table.	Elements stored in linked lists at each index.
Collision Handling	Search for another empty slot.	Store all colliding elements in a list.
Space Requirement	May waste space if table is full.	Extra space used for pointers.
Performance	Better when load factor is low.	Better when load factor is high.
Implementation	Slightly complex.	Easy to implement.

Conclusion :

Thus, collision resolution ensures efficient storage in hashing. Open addressing uses alternate slots

and its Recursive Solution

Introduction :

The Towers of Hanoi is a classic problem used to explain recursion. It involves moving disks from one rod to another following specific rules.

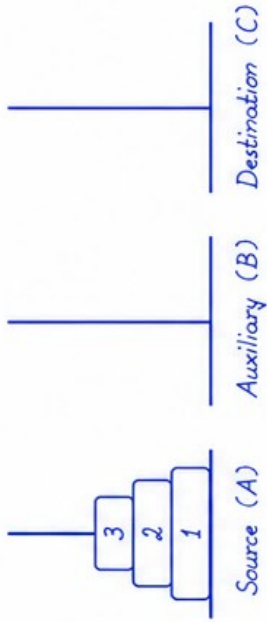
Definition (Lang) :

The Towers of Hanoi is a mathematical puzzle consisting of three rods and n disks of different sizes. The objective is to move all disks from the source rod to the destination rod using an auxiliary rod, following certain rules. It is commonly solved using recursion.

Rules of the Problem :

- Only one disk can be moved at a time.
- A larger disk cannot be placed on a smaller disk.
- Only the top disk of a rod can be moved.

Representation :



A → Source Rod

B → Auxiliary Rod

C → Destination Rod

Recursive Solution (Concept) :

To move n disks from Source (A) to Destination (C) using Auxiliary (B) :

- Move $n-1$ disks from Source (A) to Auxiliary (B).
- Move the n th (largest) disk from Source (A) to Destination (C).
- Move $n-1$ disks from Auxiliary (B) to Destination (C).

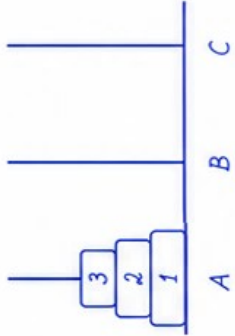
Recursive Function :

```

TOH(n, A, B, C)
{
    if (n == 1)
        move disk 1 from A to C;
    else
    {
        TOH(n-1, A, C, B);    // Step 1
        move disk n from A to C; // Step 2
        TOH(n-1, B, A, C);    // Step 3
    }
}
    
```

Example ($n = 3$) :

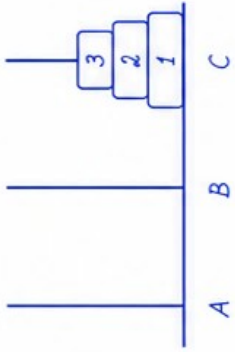
Initial State ($A \rightarrow C$)



Steps :

Step No.	Move	Action
1	$A \rightarrow C$	Move disk 1 from A to C
2	$A \rightarrow B$	Move disk 2 from A to B
3	$C \rightarrow B$	Move disk 1 from C to B
4	$A \rightarrow C$	Move disk 3 from A to C
5	$B \rightarrow A$	Move disk 1 from B to A
6	$B \rightarrow C$	Move disk 2 from B to C
7	$A \rightarrow C$	Move disk 1 from A to C

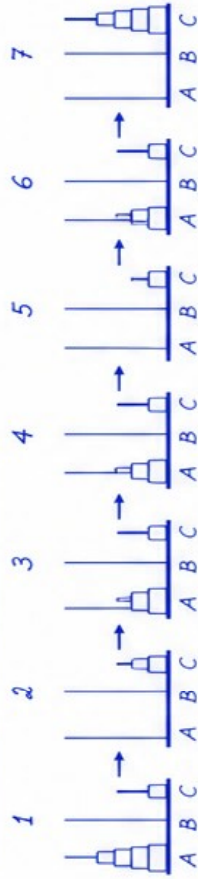
Final State (All disks at C)



Key Points :

- Total moves required = $2^n - 1$
- Uses recursion effectively
- Demonstrates divide-and-conquer approach.
- Time Complexity = $O(2^n)$
- Space Complexity = $O(n)$ (recursive call stack)

Illustration of Moves ($n = 3$) :



Conclusion :

Thus, the Towers of Hanoi problem is a classic example of recursion where a problem is broken into smaller subproblems. The recursive solution ensures that all disks are moved